

Deep Learning and Applications

Course 4

Models for sequences

Amaury Habrard

`amaury.habrard@univ-st-etienne.fr`

LABORATOIRE HUBERT CURIEN, UMR CNRS 5516
Université Jean Monnet Saint-Étienne

Today's content

- ▶ Models for sequence modeling
 - ▶ Some Problems/Applications
 - ▶ Recurrent Neural Networks (RNN)
 - ▶ Long-Short Term Memory networks (LSTM) and Gated Recurrent Unit cells.
 - ▶ Bi-directional RNN
 - ▶ A note on Word embeddings

Credits: we use the slides/resources of F. Fleuret, K. Gimpel, T.-N. Le, D. Batra, C. Verloot, S. Wiegrefe, L. Sigal.

https://www.cc.gatech.edu/classes/AY2020/cs7643_fall/

<https://fleuret.org/dlc/>

Recurrent Neural Networks (RNN)

We use slides of N. Le and D. Batra

Sequence data

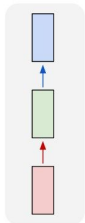


The cat ate the mouse
Le chat a mangé la souris ☺



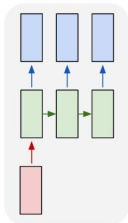
Types of Sequence Problems

one to one



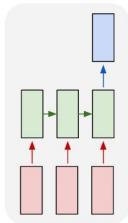
Input: No sequence
Output: No sequence
Example: "standard" classification / regression problems

one to many



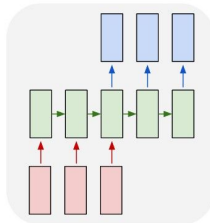
Input: No sequence
Output: Sequence
Example: Im2Caption

many to one



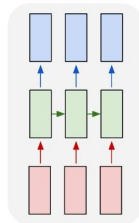
Input: Sequence
Output: No sequence
Example: sentence classification, multiple-choice question answering

many to many



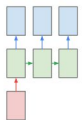
Input: Sequence
Output: Sequence
Example: machine translation, video classification, video captioning, open-ended question answering

many to many

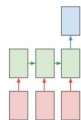


Credit: Andrej Karpathy

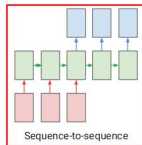
Example



One-to-sequence



Sequence-to-one



Sequence-to-sequence

Language translation is
sequence-to-sequence model

Also:

- summarization, simplification
- conversational chat-bot

Example



Two hockey players are fighting over the puck.

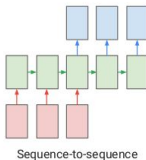
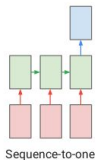
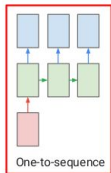
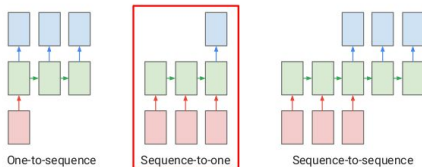
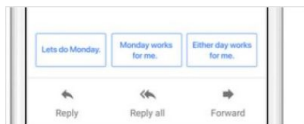


Image captioning is
one-to-sequence model

Also: Music generation

Example

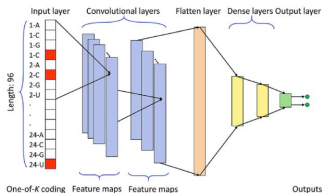


Email smart reply is
sequence-to-one model

Also:

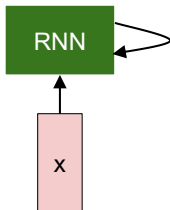
- Video activity recognition
- Sentiment classification

Limitation of CNN

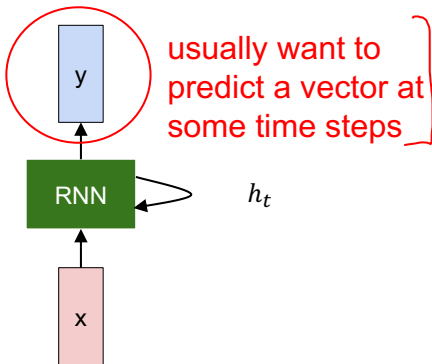


- CNN feeds the whole input sentence at once, and return a single output.
- Could work OK in case of **sequential-to-one** problems(eg. text classification)
- But not good for **sequence-to-sequence** and **one-to-sequence** problems

Recurrent Neural Network



Recurrent Neural Network



Recurrent Neural Network

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

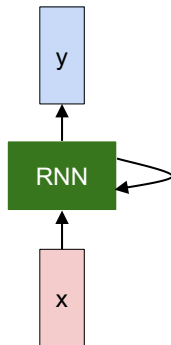
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

some function with parameters W

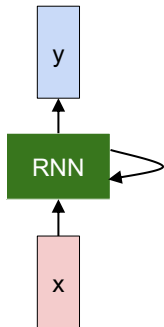
old state

input vector at some time step



(Vanilla) Recurrent Neural Network

The state consists of a single “hidden” vector $\mathbf{h}$:



$$y_t = W_{hy}h_t + b_y$$

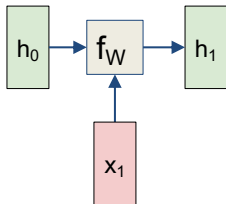
$$h_t = f_W(h_{t-1}, x_t)$$



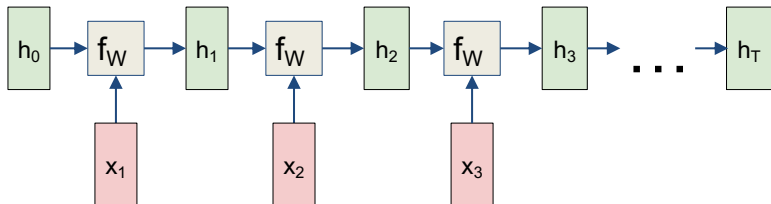
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

Sometimes called a “Vanilla RNN” or an “Elman RNN” after Prof. Jeffrey Elman

RNN: Computational Graph

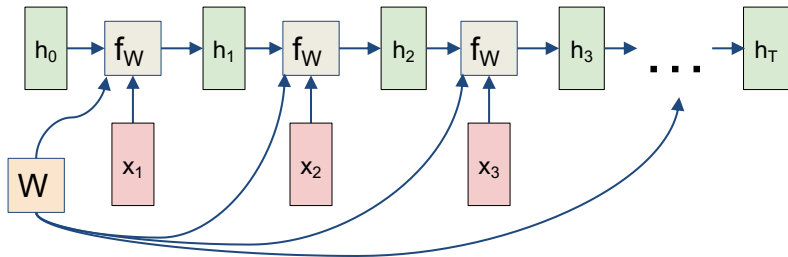


RNN: Computational Graph

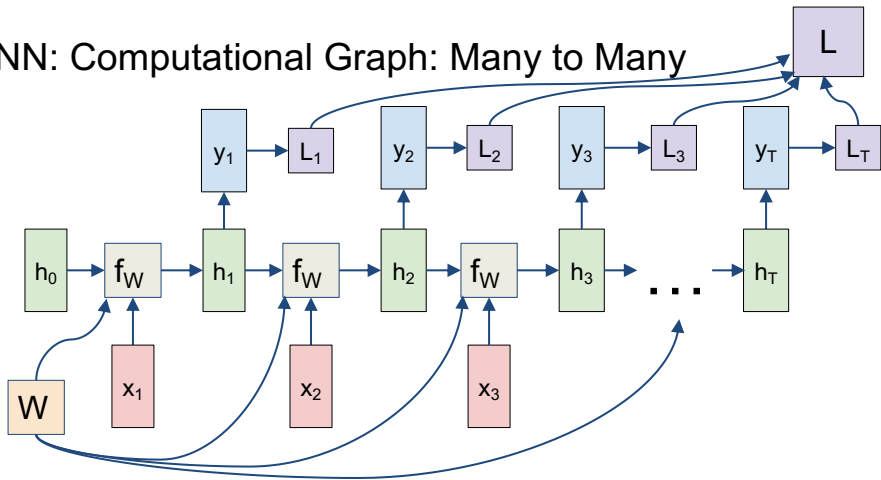


RNN: Computational Graph

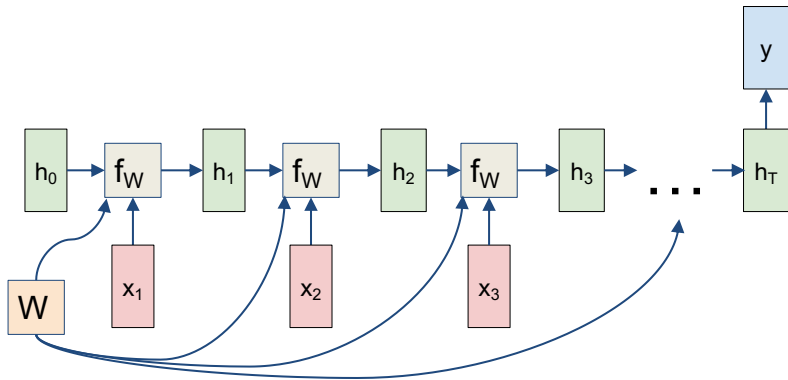
Re-use the same weight matrix at every time-step



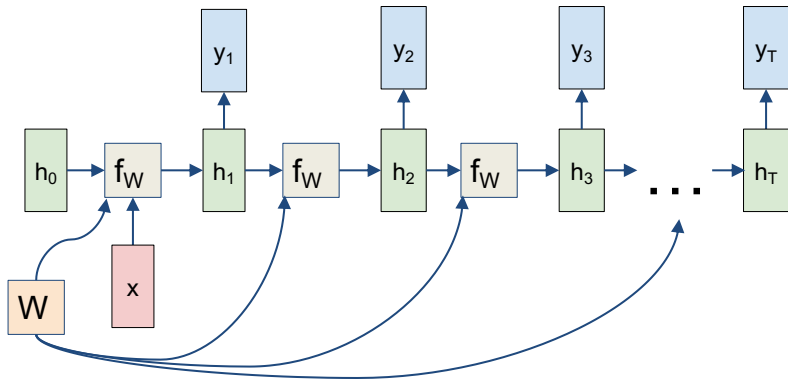
RNN: Computational Graph: Many to Many



RNN: Computational Graph: Many to One

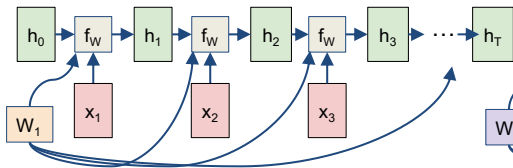


RNN: Computational Graph: One to Many

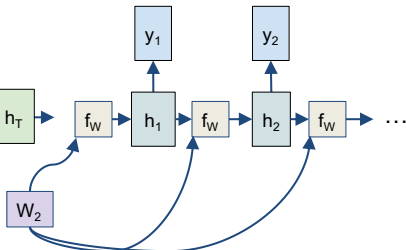


Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector



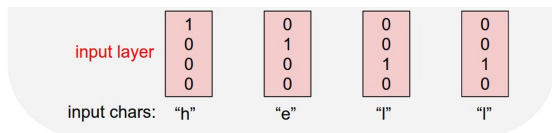
One to many: Produce output sequence from single input vector



Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

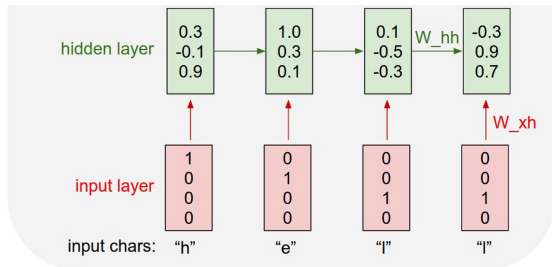


Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

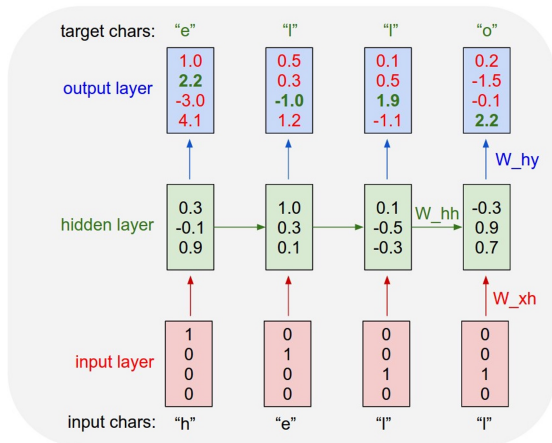
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$



Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

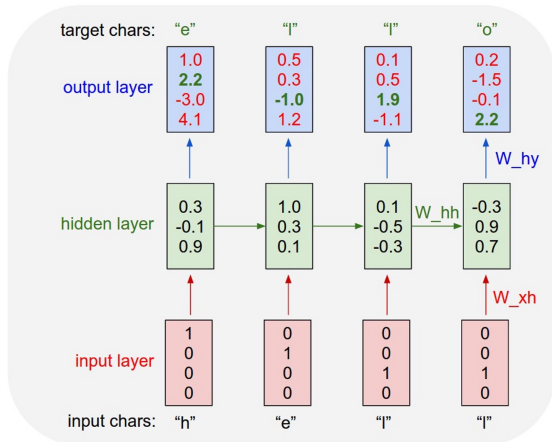


Training Time: MLE / “Teacher Forcing”

Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

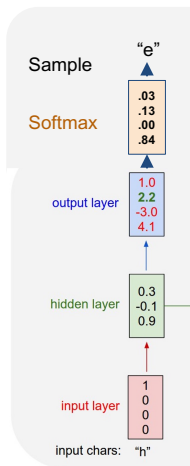


Test Time: Sample / Argmax / Beam Search

Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a
time, feed back to
model

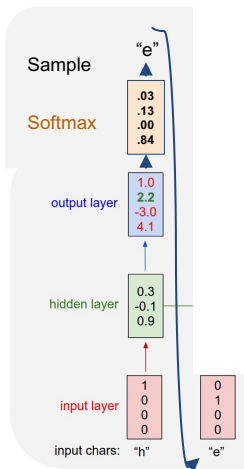


Test Time: Sample / Argmax / Beam Search

Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a
time, feed back to
model

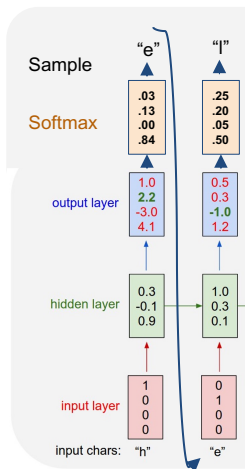


Test Time: Sample / Argmax / Beam Search

Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a
time, feed back to
model

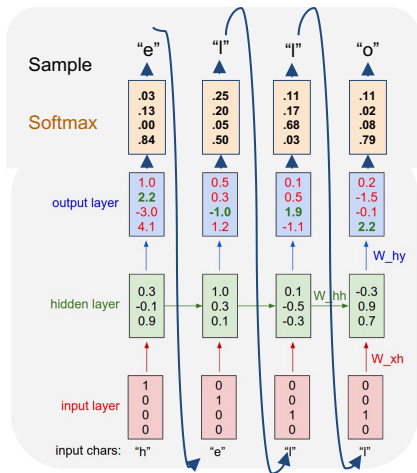


Test Time: Sample / Argmax / Beam Search

Example: Character-level Language Model Sampling

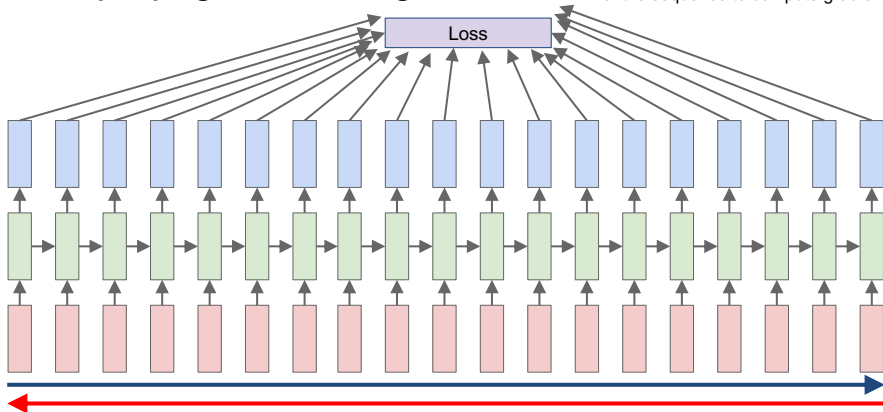
Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a
time, feed back to
model

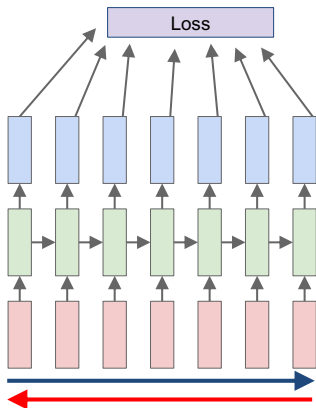


Backpropagation through time

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient

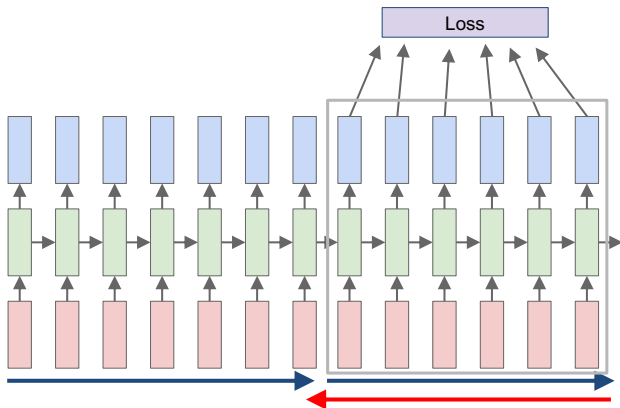


Truncated Backpropagation through time



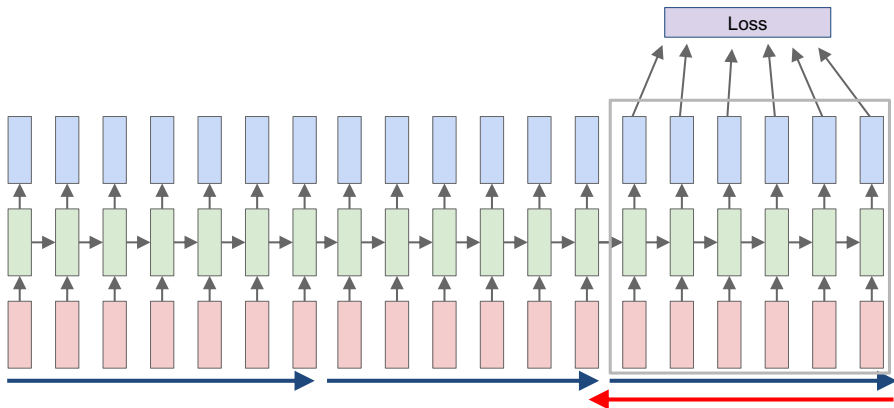
Run forward and backward through chunks of the sequence instead of whole sequence

Truncated Backpropagation through time



Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

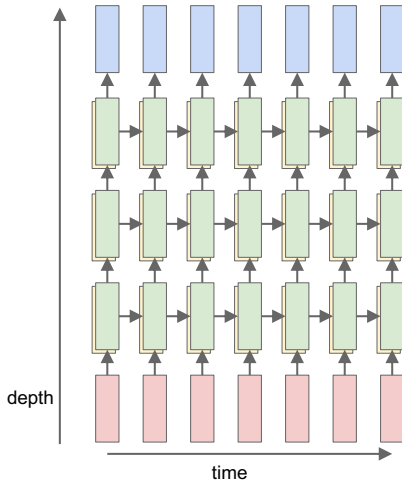
Truncated Backpropagation through time



Multilayer RNNs

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

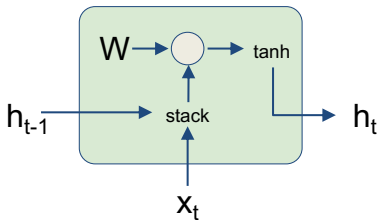
$h \in \mathbb{R}^n$ $W^l [n \times 2n]$



Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994

Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



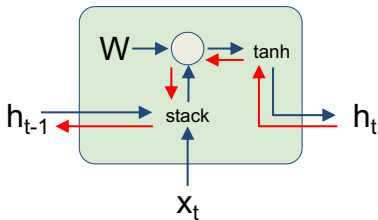
$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult". IEEE Transactions on Neural Networks. 1994

Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Backpropagation from h_t to h_{t-1} multiplies by W (actually W_{hh})

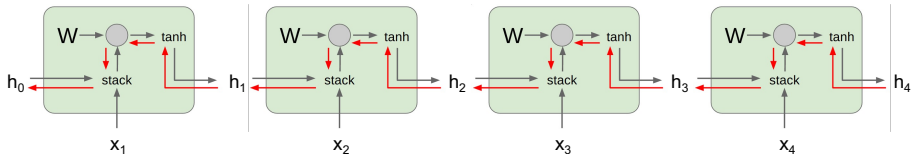


$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult". IEEE Transactions on Neural Networks. 1994

Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

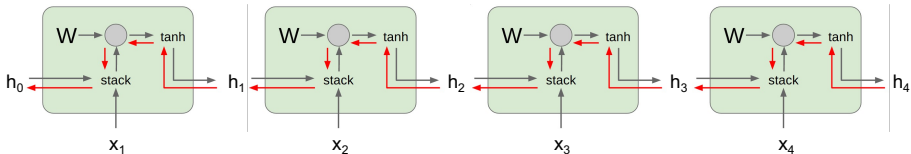


Computing gradient
of h_0 involves many
factors of W
(and repeated tanh)

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994

Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



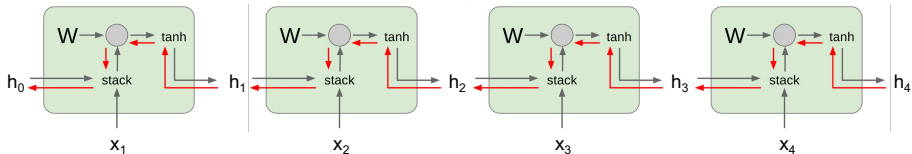
Computing gradient
of h_0 involves many
factors of W
(and repeated tanh)

Largest singular value > 1 :
Exploding gradients

Largest singular value < 1 :
Vanishing gradients

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



Computing gradient of h_0 involves many factors of W (and repeated $\tanh$)

Largest singular value > 1 :
Exploding gradients

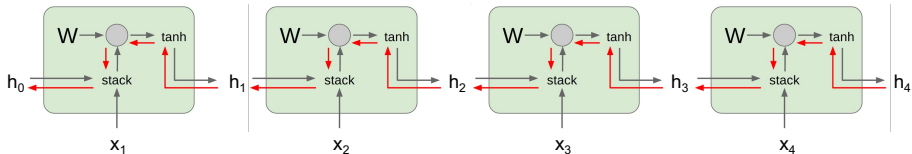
Largest singular value < 1 :
Vanishing gradients

Gradient clipping: Scale gradient if its norm is too big

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



Computing gradient of h_0 involves many factors of W (and repeated $\tanh$)

Largest singular value > 1 :
Exploding gradients

Largest singular value < 1 :
Vanishing gradients

→ Change RNN architecture

More formally Some issues with RNN (1/2)

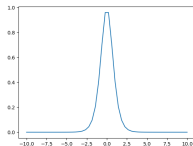
- ▶ We model a conditional distribution: $p_{\theta}(y_t | X_1, \dots, X_t)$
- ▶ We have $h_t = f_{\theta}(X_t, h_{t-1})$, the outputs are as follows:
 - ▶ $h_t = \tanh([X_t | h_{t-1}] \cdot W_h + b_h)$
or $h_t = \tanh(X_t \cdot W_x + h_{t-1} \cdot W_h + b_h)$
 - ▶ $\hat{y}_t = g_{\phi}(h_t)$
 - ▶ $W_{[x|h]}$ are the weight matrices, ϕ is a neural network mapping the learned representation h_t to the output space.
- ▶ Considering backpropagation, we have
$$\frac{\partial \mathcal{L}}{\partial \hat{y}_t} \propto \sum_{k=1}^t \frac{\partial \hat{y}_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_h}$$
- ▶ the term $\frac{\partial h_t}{\partial h_k}$ can cause trouble:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k}^{j=t-1} \frac{\partial h_{j+1}}{\partial h_j} = \prod_{j=k}^{j=t-1} (1 - \tanh^2(X_t \cdot W_x + h_{j-1} \cdot W_h + b)) W_h$$

More formally Some issues with RNN (2/2)

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k}^{j=t-1} \frac{\partial h_{j+1}}{\partial h_j} = \prod_{j=k}^{j=t-1} (1 - \tanh^2(X_t \cdot W_x + h_{j-1} \cdot W_h + b)) W_h$$

- ▶ $1 - \tanh^2$ can quickly saturates



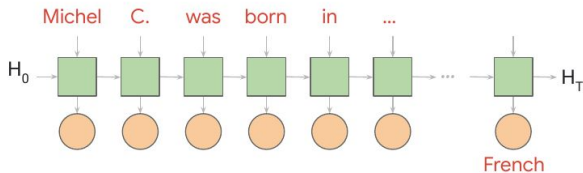
- ▶ We multiply by W_h (t-k) times, then
 - ▶ $\|W_h\|$ is high, the term increases geometrically, leading to an instability corresponding to an exploding gradient
 - ▶ $\|W_h\|$ is low, the term decreases (exponentially) towards 0, leading to vanishing gradient
- ▶ 2-layers RNN may be already heavy to train

→ need of solutions

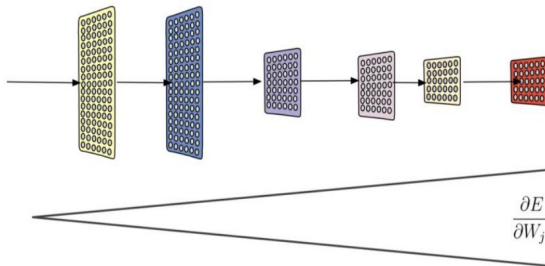
Long Short Term Memory (LSTM) and Gated Recurrent Unit (GRU) cells

Problem: Long term dependency

"Michel C. was born in Paris, France. He is married and has three children. He received a M.S. in neurosciences from the University Pierre & Marie Curie and the Ecole Normale Supérieure in 1987, and then spent most of his career in Switzerland, at the Ecole Polytechnique de Lausanne. He specialized in child and adolescent psychiatry and his first field of research was severe mood disorders in adolescent, topic of his PhD in neurosciences (2002). His mother tongue is ???"



Problem: Long term dependency

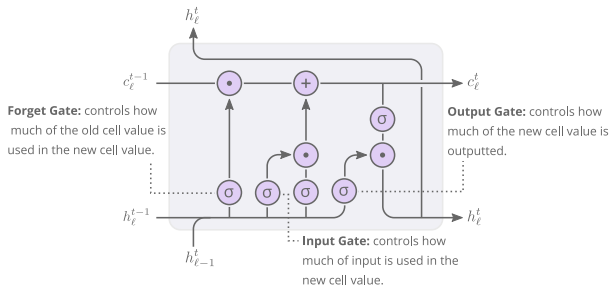


Long-term dependency causing vanishing (or sometimes, exploding) gradient in RNN (if there are 1000 timesteps -> 1000 deep layers to optimize)

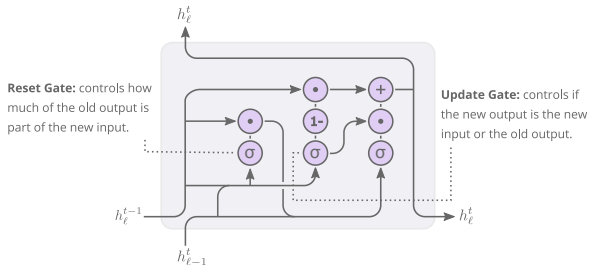
Solution: LSTM and GRU

Overviews of 2 main architectures

LSTM



GRU



Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

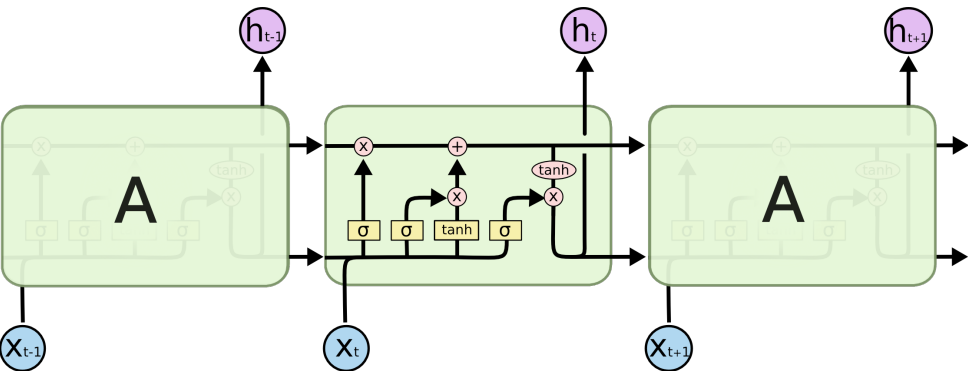
LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

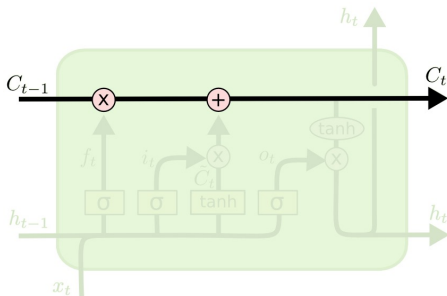
$$h_t = o \odot \tanh(c_t)$$

Meet LSTMs



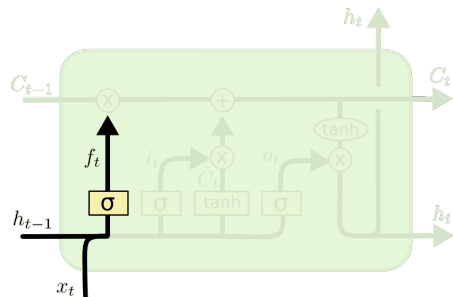
LSTMs Intuition: Memory

- Cell State / Memory



LSTMs Intuition: Forget Gate

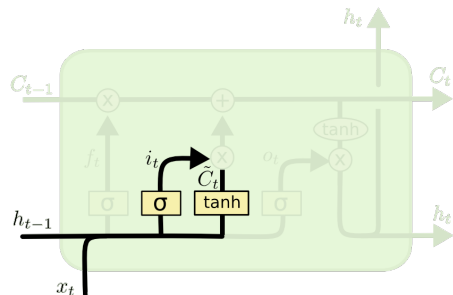
- Should we continue to remember this “bit” of information or not?



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

LSTMs Intuition: Input Gate

- Should we update this “bit” of information or not?
 - If so, with what?

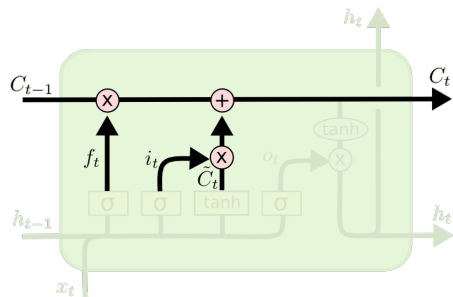


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTMs Intuition: Memory Update

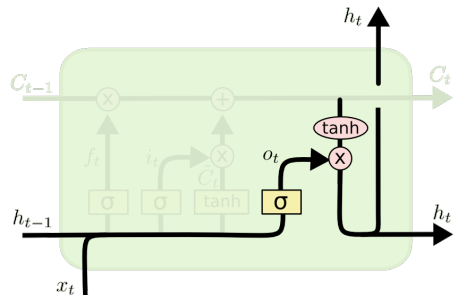
- Forget that + memorize this



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTMs Intuition: Output Gate

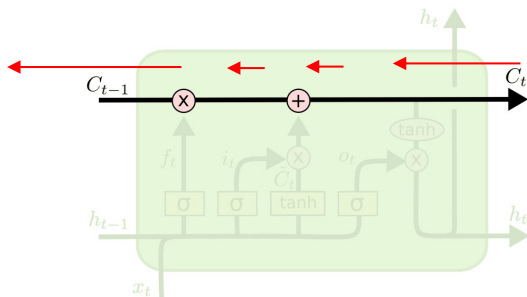
- Should we output this “bit” of information to “deeper” layers?



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

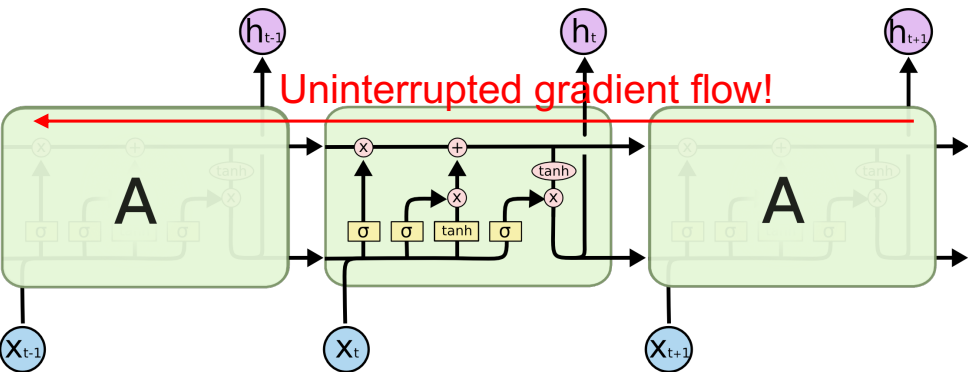
$$h_t = o_t * \tanh(C_t)$$

LSTMs Intuition: Additive Updates

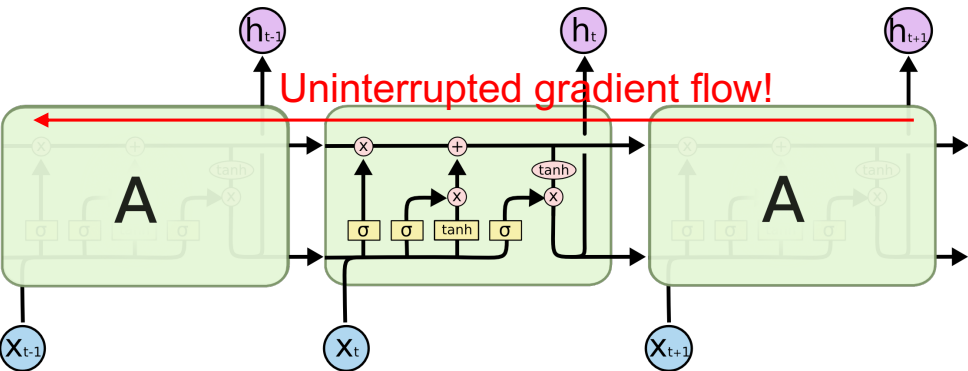


Backpropagation from c_t to c_{t-1} only
elementwise
multiplication by f , no
matrix multiply by W

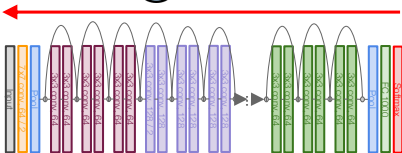
LSTMs Intuition: Additive Updates



LSTMs Intuition: Additive Updates

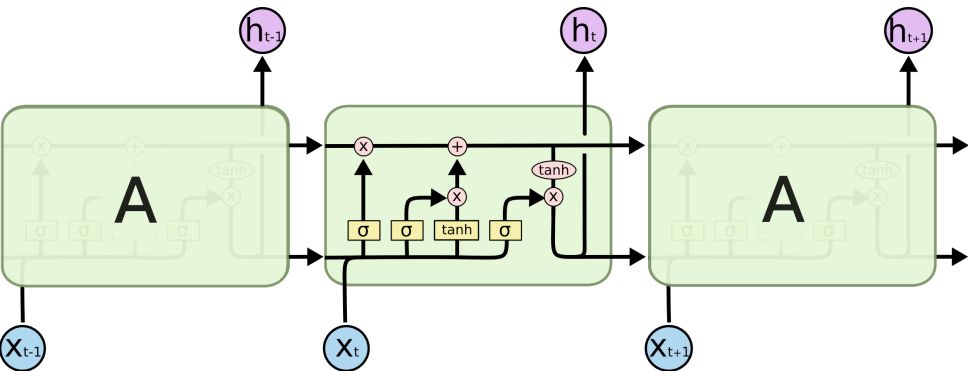


Similar to ResNet!



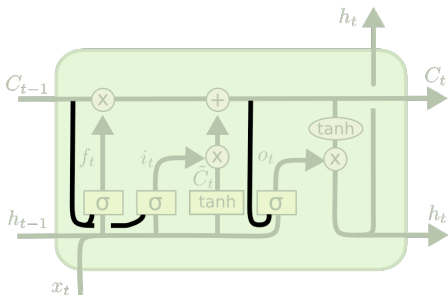
LSTMs

- A pretty sophisticated cell



LSTM Variants #1: Peephole Connections

- Let gates see the cell state / memory



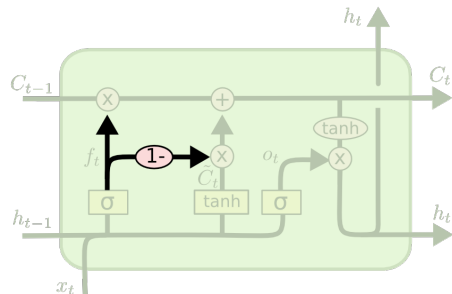
$$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$

LSTM Variants #2: Coupled Gates

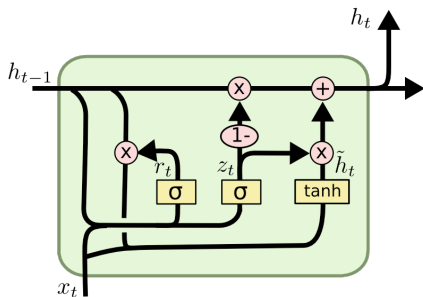
- Only memorize new if forgetting old



$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

LSTM Variants #3: Gated Recurrent Units

- Changes:
 - No explicit memory; memory = hidden output
 - Z = memorize new and forget old



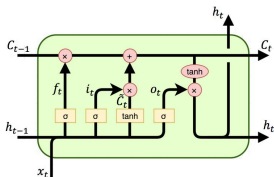
$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

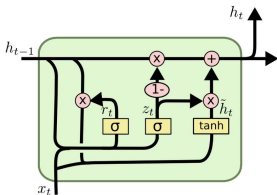
$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

LSTM vs GRU



- GRU has 2 gates, LSTM has 3 gates
- LSTM has another channel of memory unit C_i , GRU doesn't
- GRU is simpler and faster; LSTM is more powerful
- GRU is mostly on par with LSTM
- If the sequence is long, LSTM is usually better (since it has a dedicated unit C_i to remember long-distance relations)
- In practice, you should try both, and keep LSTM if it outperforms GRU



Bi-directional RNN

Problem with standard RNN

1. I am _____
2. I am _____ hungry.
3. I am _____ hungry, I could eat half a pig.

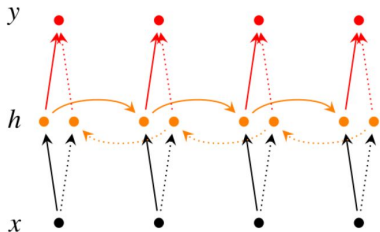
In first case: we can fill in: happy/hungry

In second case: we can fill in: not/very

In third case: only the word very is a good choice

In case 3: standard RNN only see "I am" -> doesn't work well

Bi-directional RNN



$$\vec{h}_t = f(\vec{W}x_t + \vec{V}\vec{h}_{t-1} + \vec{b})$$

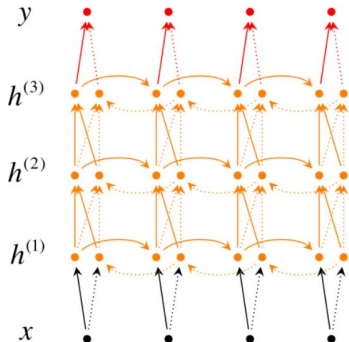
$$\overleftarrow{h}_t = f(\overleftarrow{W}x_t + \overleftarrow{V}\overleftarrow{h}_{t+1} + \overleftarrow{b})$$

$$y_t = g(U[\vec{h}_t; \overleftarrow{h}_t] + c)$$

Bi-directional RNN solve this problem:

- Have to direction run separately: left to right and right to left
- Hidden state h_t is combination of h_t forward and h_t backward, which summarizes context both side around a token

Deep bi-directional RNN



$$\vec{h}_t^{(i)} = f(\vec{W} h_t^{(i-1)} + \vec{V} \vec{h}_{t-1}^{(i)} + \vec{b}^{(i)})$$

$$\overleftarrow{h}_t^{(i)} = f(\overleftarrow{W} h_t^{(i-1)} + \overleftarrow{V} \overleftarrow{h}_{t+1}^{(i)} + \overleftarrow{b}^{(i)})$$

$$y_t = g(U[\vec{h}_t^{(L)}; \overleftarrow{h}_t^{(L)}] + c)$$

Bi-directional RNN

- Bi-RNN combined with LSTM is commonly used
- Disadvantage: Bi-RNN needs to see the whole sentence before processing
-> Doesn't work on online learning

A quick tour on Word embeddings

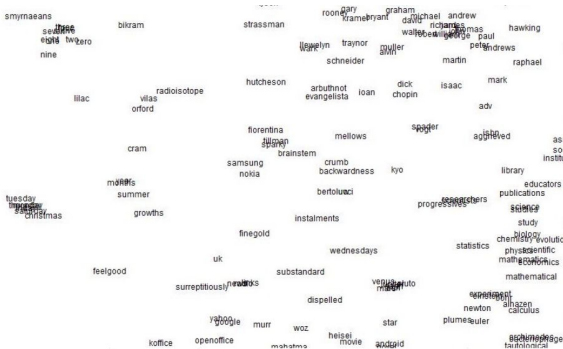
Bag of words

Bag of words and n-grams are not used for CNN and RNN:

- They almost don't take into account the order of words
- The representation vector has high dimension and is too sparse
- They represent the whole sentence in a single vector (we want to have a representation for each token to feed to the RNN at each timestep)

Solution: Represent tokens by **low-dimensional** and **dense** vectors

Word embedding



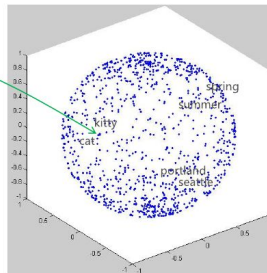
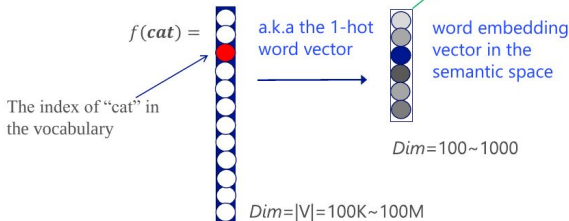
- Represent every word in a vector space (dimension 50-1000)
- Two similar words are near each other

Word embedding

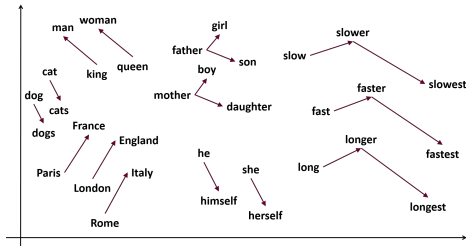
Project raw text into a continuous semantic space

e.g., word embedding

Captures the word meaning in a semantic space



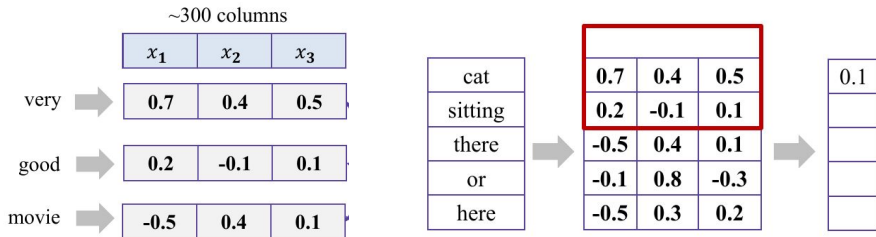
Word embedding



Relational similarity: Similar pairs of words (man-woman), (king-queen) also have similar distance:

$$\text{woman} = \text{queen} - \text{king} + \text{man}$$

Word embedding



For every sentence, we obtain a dense 2D matrix, suitable as input for both CNN and RNN

Pretrained word-embedding (over billions of sentences) is available

Representing **Phrases**: Bag-of-Words

bag-of-words representation

person holding dog {3, 4, 1} [1, 0, 1, 1, 0, 0, 0, 0, 0, 0]

person holding cat {3, 4, 2} [1, 1, 0, 1, 0, 0, 0, 0, 0, 0]

person using computer {3, 7, 6} [0, 0, 0, 1, 0, 1, 1, 0, 0, 0]

dog cat
person
holding
tree
computer
using

person using computer
person holding cat {3, 3, 7, 6, 2} [0, 1, 2, 1, 0, 1, 1, 0, 0, 0]

Vocabulary

dog	1
cat	2
person	3
holding	4
tree	5
computer	6
using	7

*slide from V. Ordonex

Distributional Hypothesis [Lenci, 2008]

- At least certain aspects of the meaning of lexical expressions depend on their distributional properties in the linguistic contexts
- The degree of semantic similarity between two linguistic expressions is a function of the similarity of the two linguistic contexts in which they can appear

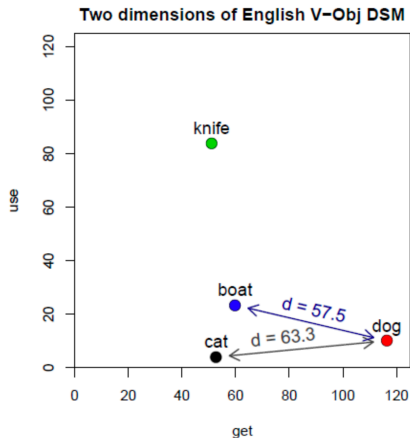
* Adopted from slides by Louis-Philippe Morency

The **Use Theory** of Meaning

“If you can understand and predict in which context a word will appear in, then you understood the meaning of the word” [Paul Horwich]

Distance and Similarity

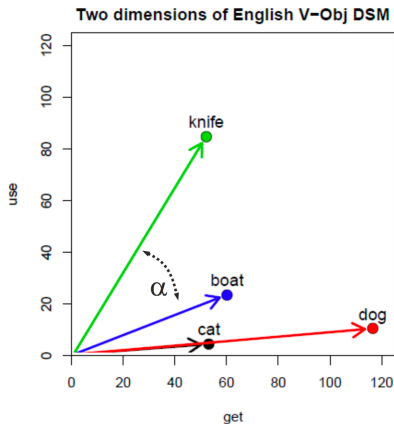
- Illustrated in two dimensions
- Similarity = spatial proximity (Euclidian distance)
- Location depends on frequency of noun (dog is 27 times as frequent as ca)



* Slides from Louis-Philippe Morency

Angle and Similarity

- direction is more important than location
- normalize length of vectors
- or use angle as a distance measure



* Slides from Louis-Philippe Morency

Geometric Interpretation: Co-occurrence as feature

- Row vector describes usage of word in a corpus of text
- Can be seen as coordinates of the point in an n-dimensional Euclidian space

Way too high dimensional!



	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	115	83	10	42	33	17
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
banana	11	2	2	0	18	0

Co-occurrence Matrix

* Slides from Louis-Philippe Morency

word2vec: Representing the Meaning of Words [Mikolov et al., 2013]

Key idea: Predict surrounding words
of every word

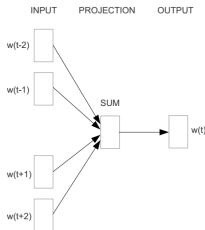
Benefits: Faster and easier to
incorporate new document, words, etc.

*slide from Vagelis Hristidis

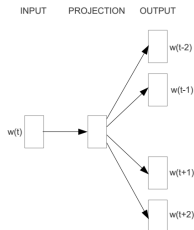
word2vec: Representing the Meaning of Words [Mikolov et al., 2013]

Key idea: Predict surrounding words of every word

Benefits: Faster and easier to incorporate new document, words, etc.



CBOW



Skip-gram

Continuous Bag of Words (**CBOW**): use context words in a window to predict middle word

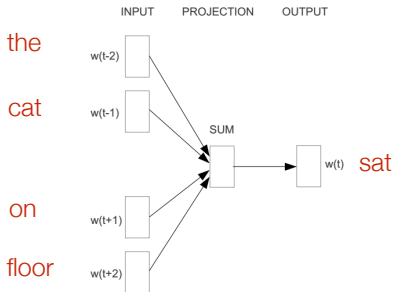
Skip-gram: use the middle word to predict surrounding ones in a window

*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

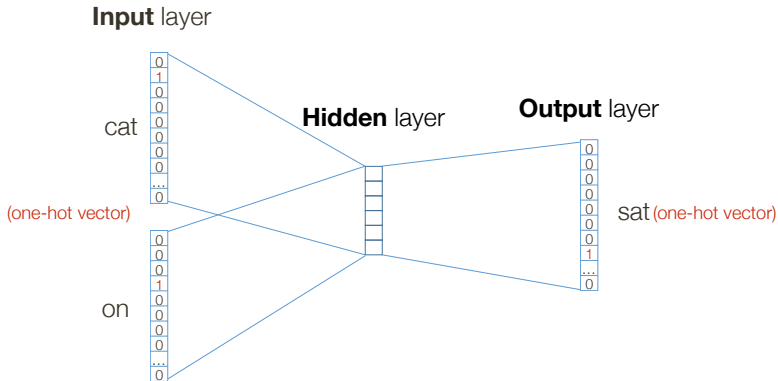
Example: “The cat sat on floor” (window size 2)



*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

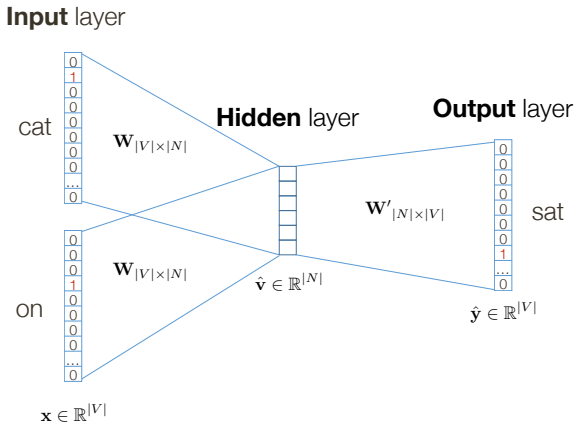
[Mikolov et al., 2013]



*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]



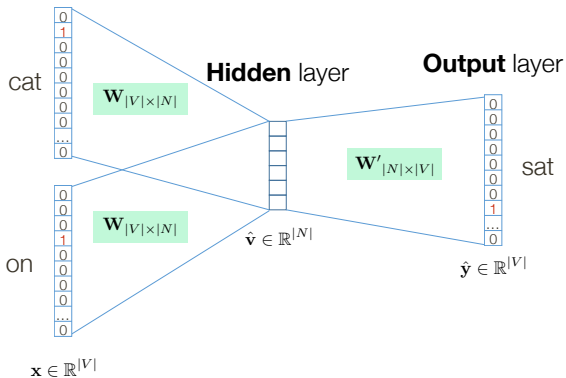
*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

Parameters to be learned



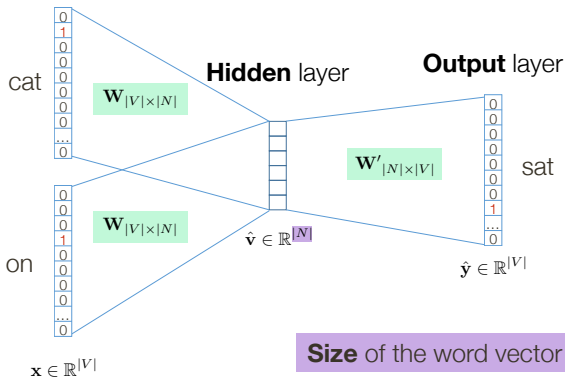
*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

Parameters to be learned

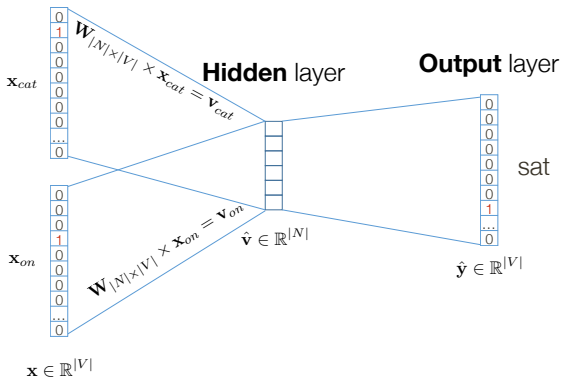


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

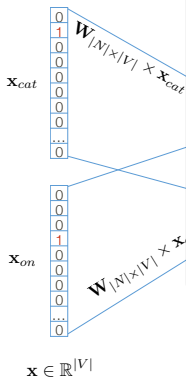


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer



$$\mathbf{W}_{|V| \times |N|}^T \times \mathbf{x}_{cat} = \mathbf{v}_{cat}$$

0.1	2.4	1.6	1.8	0.5	0.9	...	...	...	3.2
0.5	2.6	1.4	2.9	1.5	3.6	...	...	...	6.1
...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...
0.6	1.8	2.7	1.9	2.4	2.0	...	...	...	1.2

$$\times$$

0
1
0
0
0
0
0
0
0
0

$$=$$

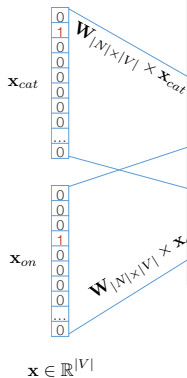
2.4
2.6
...
...
1.8

*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer



$$\mathbf{W}_{|V| \times |N|}^T \times \mathbf{x}_{on} = \mathbf{v}_{on}$$

0.1	2.4	1.6	1.8	0.5	0.9	...	...	...	3.2
0.5	2.6	1.4	2.9	1.5	3.6	...	...	...	6.1
...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...
0.6	1.8	2.7	1.9	2.4	2.0	...	...	...	1.2

$$\times \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

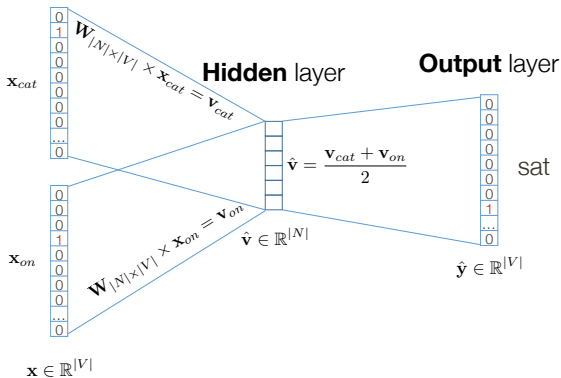
The diagram shows the calculation of the output vector $\mathbf{v}_{on}$ for the target word. The weight matrix $\mathbf{W}_{|V| \times |N|}^T$ is multiplied by the target vector $\mathbf{x}_{on}$. The resulting vector $\mathbf{v}_{on}$ is shown on the right, with values 1.8, 2.9, ..., 1.9.

*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

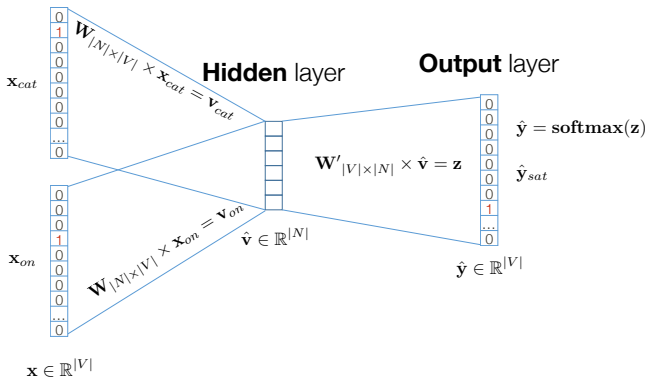


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

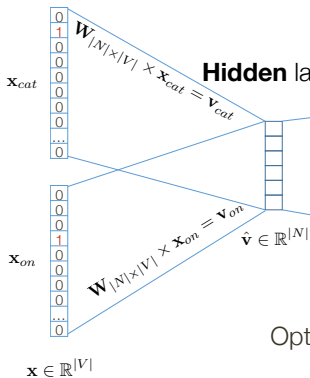


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

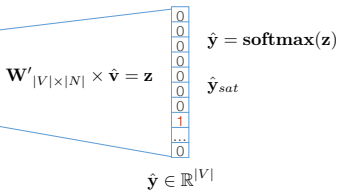
[Mikolov et al., 2013]

Input layer



Hidden layer

Output layer



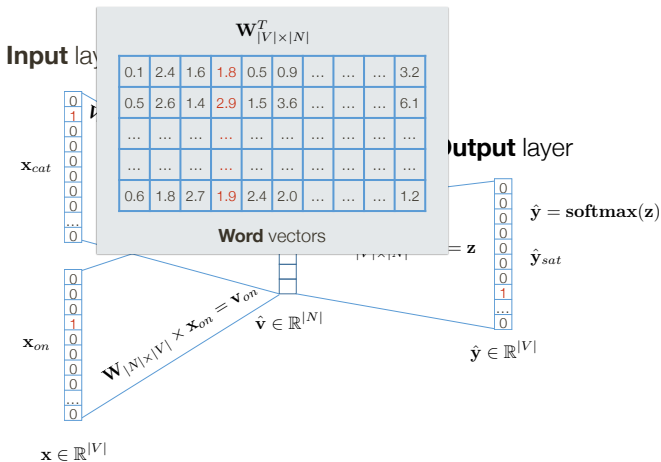
0.01
0.02
0.00
0.02
0.01
0.02
0.01
0.02
0.01
0.7
...
0.00

Optimize to get close to 1-hot encoding

*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]



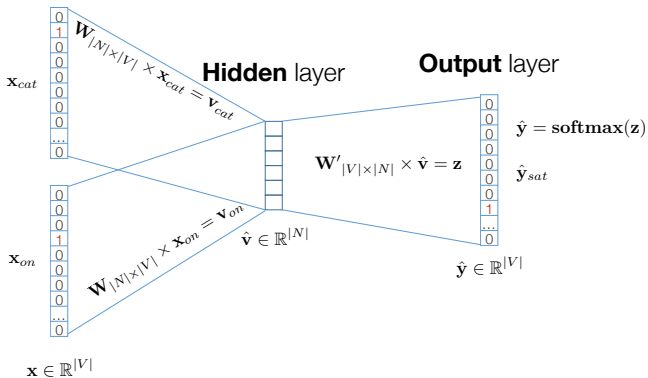
*slide from Vagelis Hristidis

CBOW: Interesting Observation

[Mikolov et al., 2013]

Input layer

There are two representations for same word!



*slide from Vagelis Hristidis

CBOW: Interesting Observation

[Mikolov et al., 2013]

Another way to look at it: Maximize similarity between context word representation and the word representation itself

$$p(w|c) = \frac{\exp \left[(\sum_c \mathbf{W}\mathbf{x}_c)^T (\mathbf{W}\mathbf{x}_w) \right]}{\sum_i^{|V|} \exp \left[(\mathbf{W}\mathbf{x}_i)^T (\mathbf{W}\mathbf{x}_w) \right]}$$

CBOW: Interesting Observation

[Mikolov et al., 2013]

Another way to look at it: Maximize similarity between context word representation and the word representation itself

$$J(\mathbf{W}) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m; j \neq 0} \log p(w_{t+j} | w_t)$$

$$p(w_{t+j} | w_t) = \frac{\exp(\mathbf{w}_{t+j}^T \mathbf{w}_t)}{\sum_{i=1}^{|V|} \exp(\mathbf{w}_i^T \mathbf{w}_t)}$$

Negative sampling

- ▶ The problem can be turned into a binary classification problem by using negative sampling.
- ▶ Intuition, for each context word pair (w_t, w_c) , generate k negative pairs (w_n, w_c) where w is a word drawn randomly
- ▶ We can define

$$P(y = +1 | w_t, w_c) = \frac{1}{1 + \exp(-w_t \cdot w_c)} ; P(y = 0 | w_n, w_c)$$

- ▶ Loss function for one positive pair:

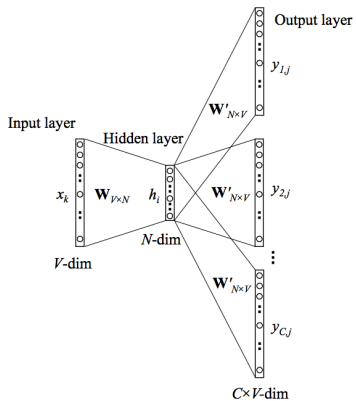
$$\log(\sigma(w_t \cdot w_c)) + \sum_{i=1}^k \mathbb{E}_{w_n, i P_n(w)} [\log \sigma(-w_n, i \cdot w_c)]$$

where $P_n(w)$ is probability distribution.

e.g. $P_n(w) = \frac{U(w)^{3/4}}{Z}$ where $U(w)$ is a unigram model, the exponent $3/4$ helps rare words to be drawn more often

Skip-Gram Model

[Mikolov et al., 2013]



Conclusion

Conclusion

- ▶ Sequential models require some specific attention to deal with vanishing exploding gradient
- ▶ Applying these models to text requires (continuous) embeddings of words
- ▶ Modeling long dependencies is a key point
- ▶ Next course will present the concept of attention and transformers which is now the state of the art of sequential and large language models.

The course is too short to see things a depth, the objective was to give you a good overview of areas, architectures and problems. You should be able to start developing your own models. Up to you to go "deeper".

Temporary page!

$\text{\LaTeX}$ was unable to guess the total number of pages correctly. If there was some unprocessed data that should have been added to the final page this extra page has been added to receive it. If you rerun the document (without altering it) this surplus page will go away, because $\text{\LaTeX}$ now knows how many pages to expect for this document.